Ship Real AI

Ship Real AI

The AI Engineer Bootcamp for LLMs, RAG, MCP,
AI Agents and Evals, from First Principles
to Production and Interviews

Sridhar Mukkandi

Draft

Ship Real AI: The AI Engineer Bootcamp for LLMs, RAG, MCP, AI Agents and Evals, from First Principles to Production and Interviews

Copyright © 2026 Sridhar Mukkandi. All rights reserved.

No part of this book may be reproduced, stored or transmitted in any form without written permission from the author, except for brief quotations in reviews and articles.

Independently published.

About the examples in this book. Tarkika, its customers and its staff are imagined. Every number about Tarkika, and every price marked as an example, is assumed, to explain the ideas clearly; none is a measurement of a real company or product. Facts about real companies, products and incidents are cited.

Products and services change quickly. Model names, versions and prices were checked on the date given in the version register at the back of the book. All trademarks belong to their respective owners.

The information in this book is provided without warranty. The author assumes no responsibility for errors or omissions, or for damages resulting from its use. Nothing in this book is legal, security-compliance or financial advice.

Text set in IBM Plex Serif, Work Sans and IBM Plex Mono, all under the SIL Open Font License.

Contact: sridhar.authorhub@gmail.com

Table of contents

Preface vi

I The job and the foundations vii

1 What an AI engineer builds 1

2 How LLMs work, for engineers 8

Appendices 19

References 19

About the author 20

Preface

The preface is written last, once the modules are final.

PART I

The job and the foundations

These modules explain what a model can and cannot do, and everything an AI engineer builds around it.

01

What an AI engineer builds

A language model can write a polite reply in Hindi, explain a return policy, or sort a thousand complaints in a few minutes. Yet when a company puts one in front of real customers, most of the work has little to do with the model itself. This module explains why. It starts with what a model can and cannot do, and ends with a map of everything an AI engineer builds around it. That map is the plan for the rest of the book.

1.1 What does a language model actually do?

You send it text, and it sends back text. That is all a single call to a model does.

GPT, Gemini, Claude, Llama, Qwen and DeepSeek are names of models. It helps to keep three words apart. The **model** is the trained program that turns text into text. The **model provider** is the company that runs it and sells access, such as OpenAI, Google or Anthropic. The **application** is a product built on top, such as the ChatGPT app. ChatGPT is not a model; it is an application that uses OpenAI's models and adds a lot around them.

Now ask a few simple questions about what a model can do on its own.

Ask it about an order placed this morning. It cannot answer. A model learns from data collected up to a certain date, and knows nothing after that. It has also never seen your company's database. This limit is called the **knowledge cutoff**.

Ask a question, then a follow-up such as "and how long will that take?" It does not know what "that" means. Every call to a model is separate, and the model keeps nothing from one call to the next. Chat apps seem to remember because the app sends the earlier messages again with every new one. Module 4 builds this.

Paste in three hundred pages of policy documents and a year of emails. It will refuse, or fail. Each call accepts only a limited amount of text, called the **context window**. Module 2 explains what fills it and why a longer input does not always give a better answer.

Ask the same question twice. You can get two different answers. An ordinary program gives the same output for the same input, every time. A model picks its words by prob-

ability, so its output can change from one call to the next (Module 2 shows how). This is the biggest difference between AI systems and ordinary software, and much of this book deals with its effects.

Ask it to check an order, send an email or issue a refund. It cannot. It can only write text. It can write "I have issued your refund", but writing that sentence does not move any money.

1.2 Three kinds of AI work

People who "work in AI" usually do one of three jobs. Some **make models**: they collect data and train models, which needs large research teams and large budgets. Some **serve models**: they make trained models run fast and cheaply on hardware, which is called inference engineering. And some **build applications**: they take a model that already exists and turn it into a product that works for real users.

This book is about the third job. You will learn enough about how models work to use them well, but you will not train a large model.

1.3 If the model cannot act, what can?

In software, what does the actual work, such as looking up an order or sending an email? Code. A function such as `check_order_status(order_id)` can query a database and return the answer.

Can that function understand a customer who writes "mera parcel kab aayega"? No. It needs an exact order number, passed in exactly the right way. The model understands the customer perfectly, but cannot query anything. Each one has exactly what the other lacks.

So put them together. The model reads the customer's message and decides which function should run and with which values. Your program runs the function. The result goes back to the model, which writes the reply (Figure 1.1). A model connected to functions it can ask your program to run is called an **agent**, and each such function is called a **tool**.

Now look at one particular kind of tool: one that fetches information, such as a search over the company's policy documents. The model asks for the relevant paragraph, the paragraph is added to the question, and the model answers using it. That is **retrieval-augmented generation**, or **RAG**. The name describes the three steps in order: *retrieve* the information, *augment* (add to) the question with it, then *generate* the answer. RAG

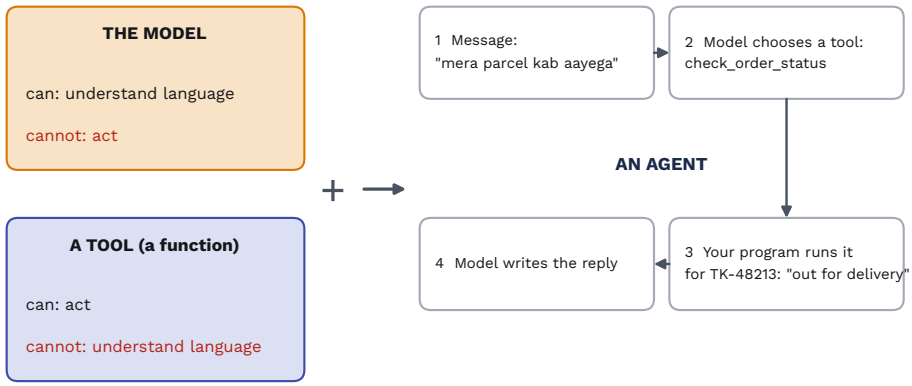


Figure 1.1 A model understands language but cannot act. A tool can act but cannot understand language. Connected, the model chooses the tool and your program runs it.

is not a separate technology. It is an agent whose tool fetches facts. Modules 6 to 8 are about doing it well.

You will also see agents described in other ways. “Model plus tools” came first. Then people noticed that what you put in front of the model decides most of the result, and started saying “model plus context”. The newest description is “model plus harness”, where the **harness** means everything around the model: the instructions, the tools, the memory, the checks, the code that runs the loop. All three descriptions are true. Building that harness is the job.

1.4 What does an AI engineer build around the model?

Figure 1.2 shows the parts, in the order a customer’s message passes through them. Each part has a module of its own.

1. **The front door** receives the message, and limits how many requests come in (Module 16).
2. **Safety checks** look at what comes in: private data, abuse, hidden instructions (Module 12).
3. **Control** decides what happens next. A fixed sequence of steps, or an agent that chooses, or a plan for a long task (Modules 11 and 14).
4. **The model** is called, perhaps several times (Modules 2 and 10).
5. **Context** decides what the model sees on each call: the instructions, the retrieved documents, the memory of the conversation (Modules 3, 4 and 6 to 8).
6. **Tools** let it act, through functions you write or through standard connectors such as MCP (Modules 5 and 13).

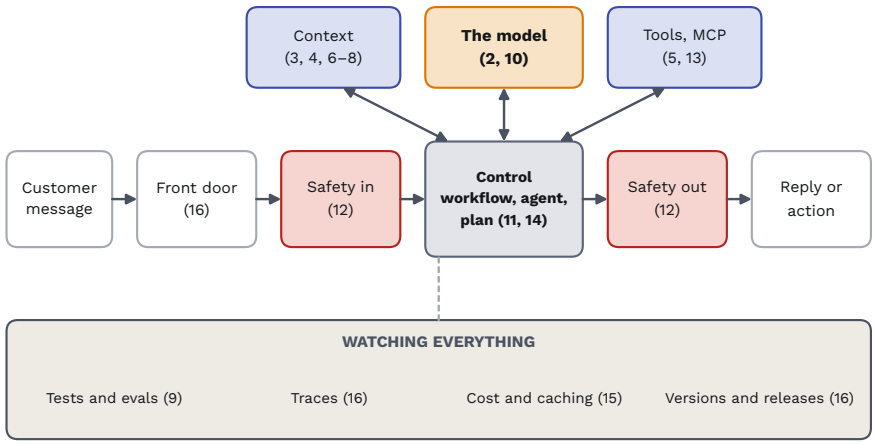


Figure 1.2 The parts of an AI system. A message enters at the front door, is checked, is handled by the control layer with the model, its context and its tools, and is checked again on the way out. Tests, traces, cost and caching watch over everything. The numbers are the modules that cover each part.

7. **Safety checks** look at what goes out, before the customer sees it or a refund is issued (Module 12).
8. Under all of it, **quality and operations**: tests that show whether it works, traces of every run, cost and caching, safe releases (Modules 9, 15 and 16).

The model is one box out of eight. The other seven are what the engineer builds.

1.5 Calling a model is not calling a function

In code, a model call looks like any other function call: text goes in, text comes out. It behaves very differently.

	An ordinary function	A model call
Speed	Microseconds	Seconds
Failure	Rarely fails	Fails sometimes: busy servers, limits, timeouts
Same input	Same output, every time	A different output is possible every time
Cost	Free to run	Charged for every call, by the amount of text

Each row changes how you write code around the model. Because the output varies, you check every reply before your code uses it (Module 5), and you cannot test it the usual way (Module 9). Because calls fail, you retry some failures and not others (Module 11). Because calls are slow, you run many at once, within the provider's limits (Module 15). And because every call costs money, you count what you send (Modules 3 and 15).

1.6 Should this problem use AI at all?

The table above gives a simple rule. A model is slower, costlier and less predictable than ordinary code. So when ordinary code can do the job, use ordinary code.

"Was this order placed more than 7 days ago?" is a date comparison. One line of code answers it correctly every time, for free. "Is this customer angry, and are they asking for a refund?", written in a mix of Hindi and English with spelling mistakes, cannot be written as a rule. That is where a model earns its cost.

Before building, ask four questions:

1. Is the input messy language, or a judgement that no rule can capture? If not, write the rule.
2. If the model is wrong, will someone notice, and how much will it cost?
3. What does success look like, as a number you can measure before and after?
4. Can you buy this ready-made? A generic support bot may be enough. You build your own when you need your own data, your own rules, or control over what it is allowed to do.

1.7 Your first call, and what it costs

Here is a complete call to a model in Python, using Anthropic's client library. Other providers' libraries look very similar.

```python
import anthropic

MODEL = "claude-opus-5-5"          # model names change; keep this in one place
client = anthropic.Anthropic()     # reads the API key from the environment

reply = client.messages.create(
    model=MODEL,
    max_tokens=1000,
    messages=[{"role": "user",
               "content": "Can I return opened earbuds after 10 days?"}],
)
for block in reply.content:
    if block.type == "text":
        print(block.text)
print(reply.usage.input_tokens, reply.usage.output_tokens)
```

The answer will be polite and sensible, and probably wrong for your shop, because the model has never seen your return policy. The last line prints how much text went in and came out, counted in **tokens**, the small pieces of text that models read and charge by. Module 2 explains what a token is.

Now work out the cost. Suppose the provider charges ₹350 for every million input tokens and ₹1,700 for every million output tokens. (These are assumed example prices. Output usually costs several times more than input, and Module 2 explains why.)

1. This question was about 30 tokens in and 120 tokens out: 30 × 350 ÷ 10,00,000 + 120 × 1,700 ÷ 10,00,000, about ₹0.21.
2. A real support request also carries instructions and the relevant part of the policy: say 3,000 tokens in and 300 out. That is 1.05 + 0.51, about ₹1.56 per message.
3. At 4,000 messages a day, that is about ₹6,240 a day, or about ₹1,87,000 a month.

Two things stand out. Most of the cost comes from the text sent *with* each question, not from the question itself. And the bill grows with every message, so a small saving per call becomes a large saving per month. Modules 3 and 15 come back to both.

1.8 The project for this book

Through the book, you will build one system: a customer-support agent for **Tarkika**, an imagined online marketplace that sells phones, electronics, fashion and home products across India. By the end, it will:

- answer questions from the company's policy documents, with the source of each answer, and say "I don't know" when the documents do not cover a question;
- look up orders, and prepare refunds that a person approves above a set amount;
- remember a customer's earlier messages, and keep each customer's data separate;
- refuse hidden instructions in customer emails;
- report its own quality, cost and speed.

Each of these is one part of Figure 1.2. The first thing to understand is the model in the middle: how it reads text, why the same question can get different answers, and why you pay for every piece of text you send. That is Module 2.

02

How LLMs work, for engineers

Module 1 treated the model as one box in a bigger system. This module opens the box, but only as far as an engineer needs. Each idea here explains something you will see in practice: why a Hindi message costs more than an English one, why the same question gets two different answers, why a long document can make answers worse, why output is priced higher than input. You will not need any maths beyond multiplication.

2.1 What does the model actually read?

A computer works with numbers, so text has to become numbers before a model can use it. How should it be split?

One way is letter by letter. Every word can then be written, but a single page becomes thousands of pieces, and each letter carries almost no meaning on its own. Another way is word by word. Each piece now means something, but there are far too many words, and every new name, spelling mistake or Hinglish word would be unknown.

Models use something in between: common pieces of words. A frequent word such as “refund” is usually one piece. A longer or rarer word is split into a few pieces, such as “un”, “believ” and “able”. Each piece is called a **token**, and the list of all pieces a model knows is fixed when the model is built. For English, one token is roughly four characters, or about three quarters of a word.

Tokens matter to an engineer for one reason above all: you pay by the token. Every price you will see is per million input tokens and per million output tokens, and every limit is counted in tokens too.

You will also hear the word **chunk**, and it is easy to confuse with a token. A token is a small piece the model reads. A chunk is a much larger piece of a document, often a few hundred tokens, that you cut out yourself so it can be searched and sent to the model. Module 7 is about chunks.

2.2 Why does a Hindi message cost more than an English one?

The list of tokens is learned from a large amount of text, and most of that text is English. So common English words get their own tokens, while words in other languages and scripts are split into many small pieces.

The same meaning in Hindi can therefore take three to five times as many tokens as in English, and a study of seventeen tokenizers found the same sentence differing by up to fifteen times between languages (Petrov et al. 2023; *Reducing Tokenization Premiums for Low-Resource Languages* 2026). Hinglish written in Latin letters usually falls in between.

More tokens for the same meaning has three effects. Each message costs more. Each message is slower, because the model works through tokens one at a time when it writes. And each message fills more of the model's input limit. Exact counts differ from model to model, so before you estimate a budget for Indian-language users, count real messages with your provider's token-counting tool.

2.3 How does the model choose the next word?

A model does one thing, again and again. It looks at all the tokens so far and gives a score to every token it knows, for how well that token would fit next. The scores are turned into chances that add up to 100%. One token is picked, added to the text, and the whole step repeats with that token included. A reply of 200 tokens is 200 of these steps.

Look at the left side of Figure 2.1. After "Your refund will reach you within", the token "7" has an 85% chance, "3" has 11%, and the rest share what is left. Most of the time "7" is picked. Sometimes "3" is.

The setting that controls how strongly the top choice is preferred is called **temperature**. At a low temperature the top token wins almost every time. At a higher temperature the chances spread out, as on the right of the figure, and the replies become more varied.

This is where Module 1's biggest difference comes from. The model is not looking up an answer. It is drawing tokens by chance, step after step, so the same question can produce a different reply each time.

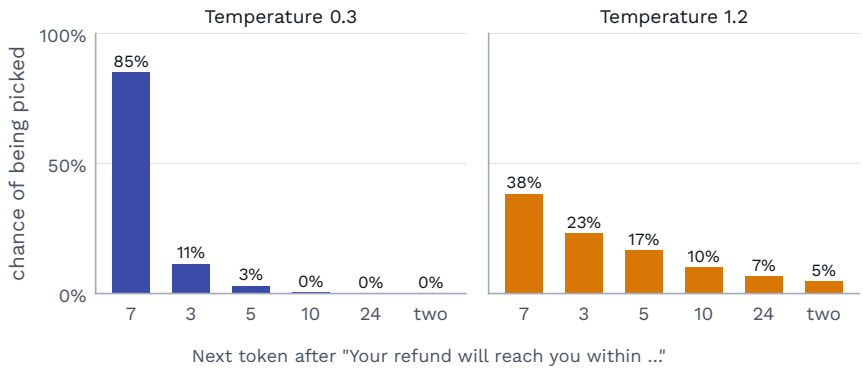


Figure 2.1 Chances for the next token at two temperatures. The model's scores are the same in both; temperature only changes how sharply they are turned into chances. (Scores made up for illustration.)

2.4 How does a network learn anything at all?

A model is a **neural network**: a very large calculation made of simple parts. One part, a **neuron**, takes some numbers, multiplies each by a weight, adds them up, and passes the result on. Neurons are arranged in layers, and each layer works on the output of the one before. A large language model has billions of these weights, which are also called **parameters**.

At the start, the weights are random and the network's guesses are useless. Training fixes that with a simple loop. Show the network some text and let it guess the next token. Measure how wrong the guess was; this number is called the **loss**. Then nudge every weight a little in the direction that would have made the guess less wrong. Repeat this a few trillion times, and the weights end up holding a great deal of what the training text contained.

The work inside is the same small calculation, multiply and add, repeated an enormous number of times. A graphics card, or **GPU**, has thousands of small cores that do exactly this calculation side by side, which is why AI runs on GPUs.

It is worth knowing the memory sums, because they decide whether you can run a model yourself. Each weight is usually stored in 2 bytes.

1. A model with 8 billion weights needs 8 × 2 = 16 GB of GPU memory just to hold them.
2. Stored at 4 bits, half a byte per weight, the same model needs about 4 GB.
3. Training it needs far more, about 16 bytes per weight once you count everything training keeps track of, so roughly 128 GB (Rajbhandari et al. 2020).

So running a model is within reach of one good GPU. Training a large one is not, which is one reason this book is about building on models rather than making them.

2.5 How does each word know what the other words mean?

Take the word “bank”. In “I sat on the river bank” it means the side of a river. In “my bank account was charged twice” it means a place for money. The token is the same in both sentences, so the token alone cannot tell the model which meaning applies. The model has to look at the words around it.

That is what **attention** does. For every token, the model works out how much each other token matters to it, and blends in information from them in that proportion. In Figure 2.2, “bank” pays most of its attention to “river”, so the model treats it as the side of a river.

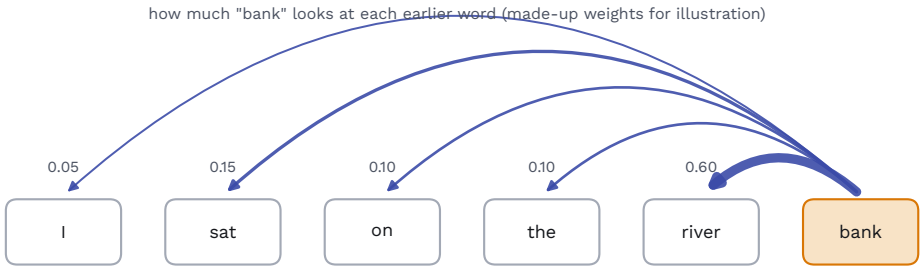


Figure 2.2 Attention: each token takes in information from the other tokens, weighted by how relevant they are. Here “bank” looks mostly at “river”. (Weights made up for illustration.)

A **transformer**, the design behind almost every language model today, is a stack of the same block repeated dozens of times. Each block has an attention step, where tokens look at each other, and a small network that works on each token on its own. The meaning of every token gets a little richer with each block.

Attention on its own ignores word order: “dog bites man” and “man bites dog” contain the same tokens. So the model also adds information about each token’s **position**.

There is one more piece that matters a great deal in practice. When the model writes token 501, attention needs information about all 500 tokens before it. Recomputing that for every new token would waste a lot of work, so the model keeps it in memory as it goes. This store is called the **KV cache**, and it grows with every token. For a typical 8-billion-weight model it takes about 128 KB per token:

1. One conversation of 8,000 tokens: 8,000 × 128 KB, about 1 GB.
2. Fifty such conversations at the same time: about 50 GB, three times the memory of the model’s own weights.

This is why long conversations and many users together are what fill a GPU, and why providers limit how much text one request may contain.

2.6 How does a model go from reading the internet to being helpful?

Training happens in stages (Figure 2.3).

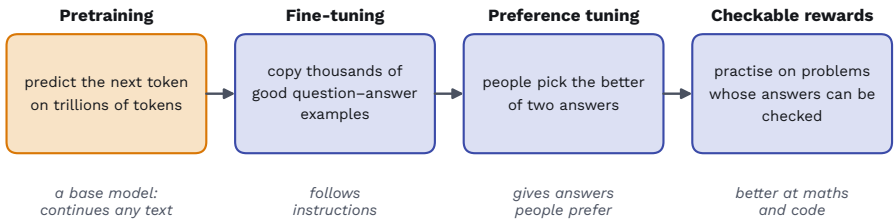


Figure 2.3 The stages of training. Each stage starts from the model the previous stage produced.

Pretraining comes first. The model learns to predict the next token on an enormous amount of text: public web pages, books, code. Modern models see trillions of tokens at this stage; Meta reports about 15 trillion for its Llama 3 models (Llama Team, AI at Meta 2024). The result is called a **base model**. It has absorbed a lot of knowledge, but it only knows how to continue text. Ask it "What is your return policy?" and it may answer with three more questions, because on the web, a question is often followed by more questions.

Fine-tuning teaches it to follow instructions. It is trained on many examples of an instruction followed by a good answer, and learns to answer instead of continuing.

Preference tuning improves the answers. People are shown two answers to the same question and pick the better one. The model is then adjusted to give answers more like the ones people preferred (Ouyang et al. 2022; Rafailov et al. 2023).

Checkable rewards are a newer stage. The model practises on problems whose answers can be checked automatically, such as maths with known answers or code with tests, and is rewarded for getting them right (DeepSeek-AI 2025). This is a large part of why recent models are so much better at maths and code than at subjects where answers cannot be checked.

Preference tuning has a side effect worth knowing. The model is rewarded for answers people *like*, and people tend to like answers that agree with them and sound confident. So models lean towards agreeing with the user and sounding sure, even when they are wrong (Sharma et al. 2023). Their stated confidence also becomes less reliable than it was before this stage (OpenAI 2023). When a model says "I am certain", treat it as a tone of voice, not a measurement. Module 10 shows how to get confidence numbers you can trust.

2.7 How can a computer tell that two sentences mean the same thing?

"I want my money back" and "refund chahiye" share no words, but mean the same thing. Searching for matching words would never connect them. What is needed is a way to turn meaning itself into numbers.

An **embedding** does this. It is a list of numbers, often a few hundred to a few thousand of them, that a special model produces for a piece of text. The model is trained so that texts with similar meanings get similar lists. Picture each list as a point in space: messages about refunds land near each other, messages about delivery land in another area, and product questions in a third (Figure 2.4).

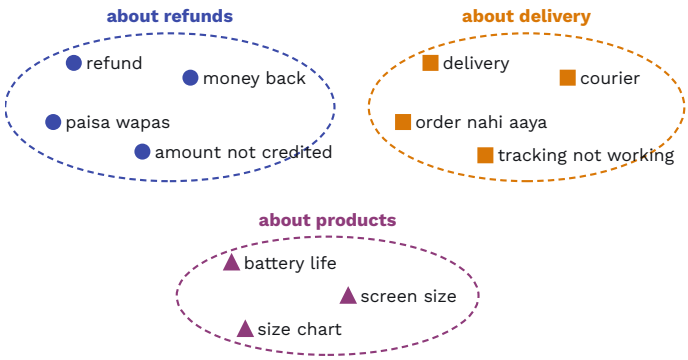


Figure 2.4 Messages placed by meaning. Real embeddings have hundreds of numbers; this picture squeezes them into two so they can be drawn. (Positions illustrative.)

To check whether two texts mean the same thing, you measure how close their points are. The usual measure is **cosine similarity**, which compares the directions of the two lists and gives a score near 1 for very similar meanings.

The length of the list is fixed by the embedding model, and it is a trade-off. More numbers can hold finer differences in meaning, but they take more space and more time to search. A million stored chunks at 1,536 numbers each, 4 bytes per number, take about 6 GB; at 384 numbers they take about 1.5 GB. There is no single right size. You choose the smallest one that still finds the right results on your own data, and Module 9 shows how to measure that.

Embedding models are separate from chat models. They only produce these lists of numbers; they do not write text. Module 6 builds search on top of them.

2.8 Why does a longer input not mean a better answer?

Every model has a **context window**: the most tokens it can take in one call, counting both what you send and what it writes. Windows have grown from a few thousand tokens to a million or more. It is tempting to send everything: the whole policy document, all past emails, every order.

Think back to attention. Each token spreads its attention across all the other tokens. When the question is 30 tokens and the material around it is 300 tokens, the question stands out. When the material is 300,000 tokens, the question is a tiny part of what the model is looking at, and the details that matter compete with a huge amount that does not.

This is what experiments find. As the input grows, accuracy falls, even on simple tasks and even far below the window's limit. This effect is called **context rot** (Hong et al. 2025). So a bigger window does not remove the need to choose what to send. It removes the excuse for not choosing. Module 3 is about choosing well.

2.9 Why does output cost more, and why is the first word the slowest?

Look at what the model does with input and with output (Figure 2.5).

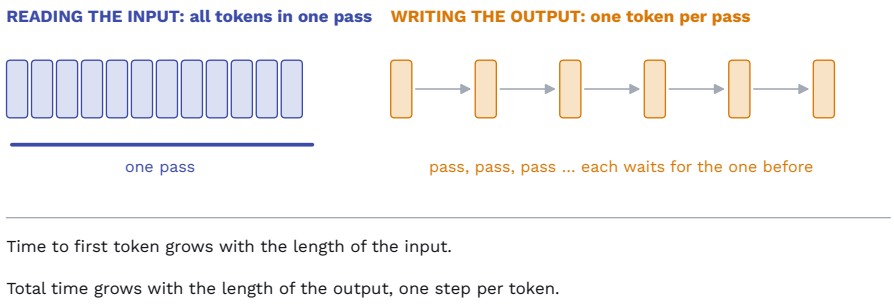


Figure 2.5 The model reads the whole input in one pass, but writes the output one token per pass.

All the input tokens are known from the start, so the model can process them together, in one large calculation that keeps the GPU busy. Output tokens cannot be produced together, because each one depends on the one before it. The model makes a full pass for every single output token, and each pass does little work compared with the GPU's capacity. So each output token uses far more GPU time than each input token, and providers charge accordingly, often around four or five times as much.

Two timings follow from this. The **time to first token** is how long the user waits before anything appears; it grows with the length of the input, because all of it must be read first. **Tokens per second** is how fast the rest arrives. For a chat window, showing tokens as they arrive, called **streaming**, makes a slow answer feel much faster, because the user starts reading early.

Model providers, and anyone who runs models themselves, use a few standard tricks to make this faster and cheaper:

- **Batching**: one pass over the GPU serves the next token for many users at once, so the idle capacity is used.
- **Prefix caching**: when many requests start with the same text, the work of reading it is done once and reused. Module 3 shows how to arrange your prompts to benefit.
- **Speculative decoding**: a small, fast model guesses the next few tokens, and the large model checks all of them in one pass, keeping those it agrees with.
- **Quantisation**: storing weights in fewer bits, such as 8 or 4 instead of 16. The model needs less memory and runs faster, for a small loss in quality that you should measure.

2.10 Why does temperature 0 still give different answers?

At temperature 0, the model should always pick the top token, so the same input should always give the same output. In practice, it often does not.

The reason is in how models are served. Your request is processed in a batch together with other people's requests, and the batch size changes from moment to moment with the load. A different batch size changes the order in which the GPU adds up its numbers. Adding the same numbers in a different order can change the last digits of the result. Usually that changes nothing. But when two tokens are almost tied, a tiny change can flip which one is on top, and from that point the rest of the reply goes a different way.

One study ran the same prompt 1,000 times at temperature 0 on a standard serving setup and got many different completions; with specially written calculations that do not depend on batch size, all 1,000 were identical, at some cost in speed (He and Thinking Machines Lab 2025). Most providers do not run that way. So do not build anything that depends on a model giving exactly the same text twice. Check properties of the answer instead, as Module 9 explains.

2.11 When does it pay for a model to think first?

Answer 2 + 2 and you need no working. Answer 17 × 24 in your head and you probably break it into steps. Models are similar. For a problem with several steps, they do better if they first write out intermediate reasoning and only then give the answer.

Thinking models do this on their own. Before the visible answer, they produce reasoning tokens, and a setting usually called **effort** controls how much. Those reasoning tokens are output tokens, and you pay for them like any other.

Do the sum for a simple job: sorting a message into one of four categories.

1. Without thinking, the answer is about 20 output tokens.
2. With heavy thinking, the model might write 1,500 tokens of reasoning first, so about 1,520 output tokens in all.
3. That is 76 times the output cost, for the same one-word label.

For sorting messages, that is waste. For working out a refund across three partial returns and a coupon, the reasoning may be what makes the answer right. So compare models by the **cost per finished task**, including retries and mistakes, not by the price per token.

2.12 How can a model be very large and still fast?

Larger models generally know more and reason better, but each token passes through every weight, so the work per token grows with the model's size.

A **mixture of experts** gets around this. The network is divided into many parallel parts, called **experts**, and a small **router** sends each token to only a few of them (Figure 2.6). The model can have a very large total number of weights, while each token uses only a small share. Mixtral 8x7B, for example, has about 47 billion weights in total, but uses about 13 billion for each token (Jiang et al. 2024).

For an engineer, the important point is that **speed follows the weights used per token, but memory follows the total**. All the experts must be loaded, because the next token may need different ones. A mixture-of-experts model can run as fast as a small model and still need the memory of a large one.

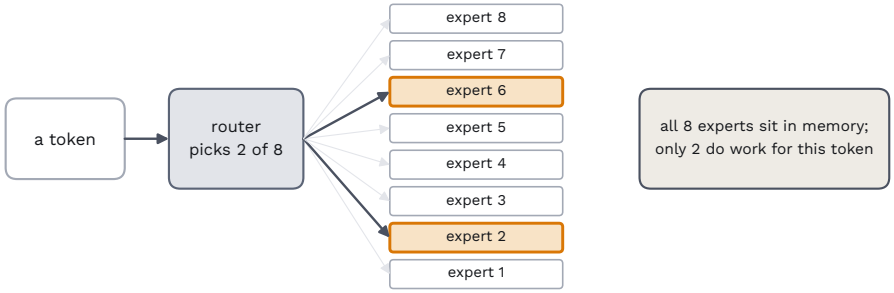


Figure 2.6 A mixture of experts. A router sends each token to a few experts; the rest are idle for that token but must still be held in memory.

2.13 How do models read images, documents and speech?

The same design extends beyond text. An image is cut into small square patches, and each patch is turned into a list of numbers, much like a token. From then on, the transformer handles the patches together with the text tokens, so it can answer a question about a photo. A scanned document or PDF page can be read the same way, as an image, or its text can be extracted first. Speech is cut into short slices of sound and handled the same way, or turned into text first by a separate model.

Images are not free. One image can cost as many tokens as a page or more of text, so sending ten screenshots with every question adds up quickly.

2.14 Which model should do which job?

Everything in this module points to the same answer: there is no single best model, only the right model for each job.

- **Sorting, extracting and short answers** need a small, fast, cheap model. Paying for a large one gives the same label at many times the cost.
- **Reasoning over several steps, or writing that must be precise**, needs a large model, sometimes with thinking turned on.
- **Open-weight models**, whose weights you can download, can run on your own machines. That keeps data in-house and can be cheaper at high volume, but you run the GPUs, and the licence may limit commercial use. Open weights are not always free to use; read the licence.
- **Models through an API** need no hardware, and are usually the best place to start.

The only reliable way to choose is to try a few models on a set of your own real examples and compare quality, speed and cost per finished task. Module 9 shows how to build that set.

You now know what the model in the middle of the system map does, and why it behaves as it does. It knows only what is in front of it at the moment of the call. So what should be in front of it, and in what order? That is Module 3.

References

DeepSeek-AI. 2025. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948.

He, Horace, and Thinking Machines Lab. 2025. *Defeating Nondeterminism in LLM Inference*. Thinking Machines Lab: Connectionism.

Hong, Kelly, Anton Troynikov, and Jeff Huber. 2025. *Context Rot: How Increasing Input Tokens Impacts LLM Performance*. Chroma technical report. https://research.trychroma.com/context-rot.

Jiang, Albert Q., Alexandre Sablayrolles, Antoine Roux, et al. 2024. *Mixtral of Experts*. arXiv:2401.04088.

Llama Team, AI at Meta. 2024. *The Llama 3 Herd of Models*. arXiv:2407.21783.

OpenAI. 2023. *GPT-4 Technical Report*. arXiv:2303.08774.

Ouyang, Long, Jeff Wu, Xu Jiang, et al. 2022. 'Training Language Models to Follow Instructions with Human Feedback'. *Advances in Neural Information Processing Systems (NeurIPS)*.

Petrov, Aleksandar, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. 2023. 'Language Model Tokenizers Introduce Unfairness Between Languages'. *Advances in Neural Information Processing Systems (NeurIPS)*.

Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. 'Direct Preference Optimization: Your Language Model Is Secretly a Reward Model'. *Advances in Neural Information Processing Systems (NeurIPS)*.

Rajbhandari, Samyam, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. 'ZeRO: Memory Optimizations Toward Training Trillion Parameter Models'. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*.

Reducing Tokenization Premiums for Low-Resource Languages. 2026. arXiv:2601.13328. https://arxiv.org/abs/2601.13328.

Sharma, Mrinank, Meg Tong, Tomasz Korbak, et al. 2023. *Towards Understanding Sycophancy in Language Models*. arXiv:2310.13548.

About the author

Sridhar Mukkandi is an applied AI engineer who builds AI agents and the decision systems behind them. He writes for engineers who want AI systems they can trust in production.

sridhar.authorhub@gmail.com